hashlock.

2024

Security Audit

YKYR (DLP)

December 2024 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

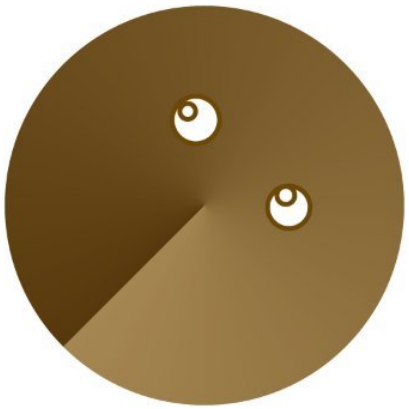
Executive Summary

The YKYR team partnered with Hashlock to conduct a security audit of their YKYRDLP.sol, IYKYRDLP.sol, YKYRDLPDefs.sol, YKYRDLPStorageV1.sol, YKYRWalletImp.sol, YKYRWalletFactory.sol and IYKYRWalletImp.sol smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

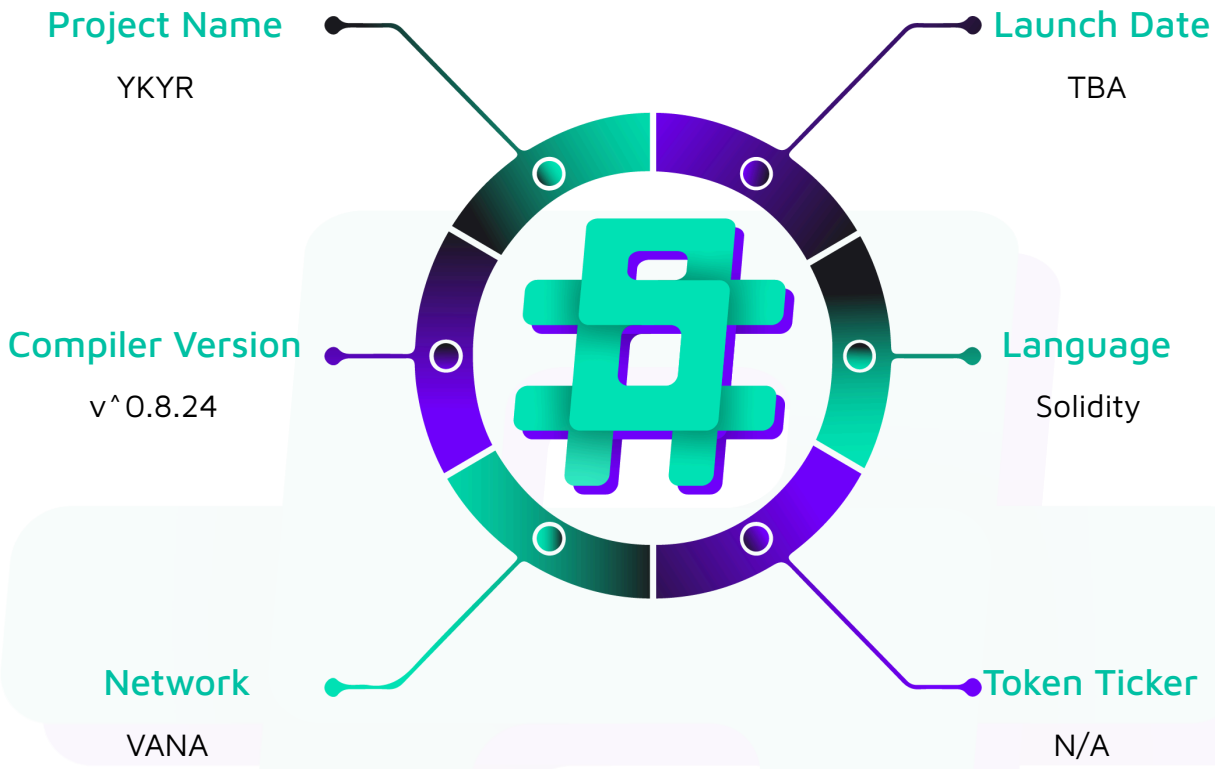
Project Context

YKYR empowers users to collect, own, and monetize their personal browsing data and behavioral traits. By allowing individuals to gather unique datasets derived from their online activities, YKYR provides a platform where users can profit from their data through a decentralized marketplace or utilize it to train custom AI models tailored to their preferences.

Project Name: YKYR
Compiler Version: ^0.8.24
Website: www.ykyr.org
Logo:



Visualised Context:



Project Visuals:

Audit scope

We at Hashlock audited the solidity code within the YKYR project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	YKYR Protocol Smart Contracts
Platform	Vana / Solidity
Audit Date	December, 2024
Contract 1	YKYRDLP.sol
Contract 1 MD5 Hash	5b3e6fb4732b0cf917fc2e2a69b59bb7
Contract 2	YKYRDLPDefs.sol
Contract 2 MD5 Hash	3dc0aa2059f991a6f9fac4ec5c770e52
Contract 3	YKYRDLPStorageV1.sol
Contract 3 MD5 Hash	11e8a08837d9127185ce2887e073814b
Contract 4	IYKYRDLP.sol
Contract 4 MD5 Hash	b438f89cd1433c7aa6cee09da9179e4a
Contract 5	YKYRWalletImp.sol
Contract 5 MD5 Hash	496b0ae8df7ec2c6c909835c4e503a4a
Contract 6	YKYRWalletFactory.sol
Contract 6 MD5 Hash	5862d9c4919542925e3405a1c0475c13
Contract 7	IYKYRWalletImp.sol
Contract 7 MD5 Hash	06cb5ac8ecc4665c97adadb9774aed7d
GitHub Commit Hash	db3e42cc3282261a9199b54caba943fdc31767d5

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The general security overview is presented in the Standardised Checks section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

1 Medium severity vulnerabilities

7 Low severity vulnerabilities

6 Gas Optimisations

3 QA findings

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

YKYRDLP.sol
- Allows users to:
  - Register as contributors either directly or through a relayer
  - Request rewards for submitted files with valid proofs
  - Create personal YKYR wallets to manage their contributions
- Allows admins to:
  - Update protocol parameters (reward factors, master keys, proof instructions)
  - Manage integrations with other contracts (DataRegistry, TeePool)
  - Pause/unpause the contract
  - Add rewards to the contributor pool
	Contract achieves this functionality.

IYKYRDLP.sol - Interface contract - Defines all external functions and events that YKYRDLP must implement	**Contract achieves this functionality.**
YKYRDLPDefs.sol - Shared data structures library used across the protocol - Defines core structures like File, FileResponse, Contributor - Provides standardized response types for protocol interactions	**Contract achieves this functionality.**
YKYRWalletImp.sol	**Contract achieves this**

- Allows users to:
  - Auto-submit files to the DataRegistry
  - Submit jobs to the TeePool
  - Manage proxy and relayer permissions
  - Set reward collection addresses
  - Claim accumulated rewards
- Allows admins to:
  - Update integration contracts (DataRegistry, TeePool)
	functionality.

YKYRWalletFactory.sol
- Allows users to:
  - Create new wallet instances either directly or through a relayer
  - Deploy minimal proxy clones of the wallet implementation
- Allows admins to:
  - Set the implementation contract address
	Contract achieves this functionality.

YKYRWalletImp.sol - Interface contract - Defines core wallet functionality including	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the YKYR project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the YKYR project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Audit Findings

High

[H-01] YKYRDLP#requestReward - The same `fileId` can be used repeatedly when the reward amount is 0 which will cause manipulation of state variables

Description

The `requestReward` function allows users to receive rewards for the files they have presented into the protocol. However, this function allows the same file to be processed multiple times when the file's proof score is 0, leading to inflated contributor counts, corrupted contributor tracking in the contract's state and fault event emissions.

Vulnerability Details

The `requestReward` function allows users to receive rewards for the files they have presented into the protocol.

```
      function requestReward(uint256 fileId, uint256 proofIndex) external override whenNotPaused nonReentrant {

              IDataRegistry.Proof memory fileProof = _dataRegistry.fileProofs(fileId, proofIndex);


                                if (keccak256(bytes(fileProof.data.instruction)) != keccak256(bytes(_proofInstruction))) {

            revert InvalidProof();

        }


        YKYRDLPDefs.File storage file = _files[fileId];


        if (file.rewardAmount != 0) {

            revert FileAlreadyAdded();

        }
```

```
        IDataRegistry.FileResponse memory registryFile = _dataRegistry.files(fileId);

        bytes32 messageHash = keccak256(
            abi.encodePacked(
                registryFile.url,
                fileProof.data.score,
                fileProof.data.dlpId,
                fileProof.data.metadata,
                fileProof.data.proofUrl,
                fileProof.data.instruction
            )
        );

                                                bytes32      ethSignedMessageHash      =
MessageHashUtils.toEthSignedMessageHash(messageHash);
        address signer = ECDSA.recover(ethSignedMessageHash, fileProof.signature);

        if (!_teePool.isTee(signer)) {
            revert InvalidAttestator();
        }

        // Overflow check
        require(
            fileProof.data.score <= type(uint256).max / _fileRewardFactor,
            "Multiplication overflow"
        );

        uint256 product = _fileRewardFactor * fileProof.data.score;
        file.rewardAmount = product / 1e18;

        file.timestamp = block.timestamp;
        file.proofIndex = proofIndex;
        file.rewardAmount = (_fileRewardFactor * fileProof.data.score) / 1e18;
        _filesList.add(fileId);
```

```
                                YKYRDLPDefs.Contributor    storage    contributor    =
_contributorInfo[registryFile.ownerAddress];

        contributor.filesList.add(fileId);

         // ADD the contributor.points = points; where the points value is retrieved from
the fileProof.data.metadata.points

        Address.sendValue(payable(registryFile.ownerAddress), file.rewardAmount);


        _totalContributorsRewardAmount -= file.rewardAmount;


        if (contributor.filesList.length() == 1) {

            _contributorsCount++;

            _contributors[_contributorsCount] = registryFile.ownerAddress;

        }


                  emit  RewardRequested(registryFile.ownerAddress,  fileId,  proofIndex,
file.rewardAmount);

    }
```

As observed in the highlighted part, the check that is being done to see if the file has been added before only checks if this file has received a reward already. However, this does not account for the files that have a reward score of 0 and these files can be used repeatedly.

Since the return values of `add()` calls are not checked, the same file being used will not be added but the function will also not revert, meaning that the function will keep executing since the file has not been added again, the following if check will pass.

```
        if (contributor.filesList.length() == 1) {

            _contributorsCount++;

            _contributors[_contributorsCount] = registryFile.ownerAddress;

        }
```

This results in the same file being used more than once, inflating the `_contributorsCount` amount and adding the same address more than once to the `_contributors` mapping even when the same file is used. This also causes misleading event emissions.

Proof of Concept

Add the following test in the `dlp.ts` test file and run it to observe the vulnerability.

```
describe("RequestReward Vulnerabilities", () => {
  beforeEach(async () => {
    await deploy();
    await dlp.connect(owner).addRewardsForContributors(dlpInitialBalance);
  });

  it("demonstrates vulnerability with zero score files", async function () {
    // Add a file to the registry
    await dataRegistry
      .connect(sponsor)
      .addFileWithPermissions("file1Url", user1, []);

    await teePool.connect(sponsor).submitJob(1, { value: parseEther("0.01") });

    // Create a proof with score = 0
    const zeroScoreProof: Proof = {
      signature: await signProof(tee0, "file1Url", {
        ...proofs[1].data,
        score: 0 // Set score to 0
      }),
      data: {
        ...proofs[1].data,
        score: 0
      }
    };
```

```
    await dataRegistry.connect(tee0).addProof(1, zeroScoreProof);

    // First request
    await dlp.connect(sponsor).requestReward(1, 1);
    const contributorsAfterFirst = await dlp.contributorsCount();

    // Second request with same file should succeed in vulnerable contract
    await dlp.connect(sponsor).requestReward(1, 1);
    const contributorsAfterSecond = await dlp.contributorsCount();

    // Prove the vulnerability: contributorsCount incorrectly increases
    expect(contributorsAfterSecond).to.be.gt(contributorsAfterFirst);

    // Verify the same address is counted multiple times
    const contributor1First = await dlp.contributors(1);
    const contributor1Second = await dlp.contributors(2);
    expect(contributor1First.contributorAddress).to.equal(user1);
    expect(contributor1Second.contributorAddress).to.equal(user1);
  });
});
```

Impact

Manipulation of relevant state variables and misleading event emissions. This is increasingly more problematic when these state variables are used in off-chain and future on-chain systems.

Recommendation

Implement checks in the function to check for the returned value of `add()` calls. If this call returns false, revert.

Status

Resolved

Medium

[M-01] YKYRWalletFactory#createWallet, createWalletViaRelayer - Lack of access control leads to malicious event emits that will have off-chain impact

Description

In the `YKYRDLP` contract, the `registerContributorViaRelayer` and `createYKRWallet` functions make a call to the `createWalletViaRelayer` and `createWallet` functions in the `YKYRWalletFactory` contract respectively. However, the functions in the `YKYRWalletFactory` contract lack access control that would allow them to only be called by the `YKYRDLP` contract or any whitelisted entity. This will lead to attackers front-running `registerContributorViaRelayer` and `createYKRWallet` calls and manually calling the functions in the `YKYRWalletFactory` contract, triggering an event emitted to manipulate the off-chain systems and users.

Vulnerability Details

In the `YKYRDLP` contract, the `registerContributorViaRelayer` and `createYKRWallet` functions are used to create a new wallet.

```
    function registerContributorViaRelayer(address userAddress, address relayerAddress, address proxyAddress, uint256 salt, bytes memory userSignature) public {

        require(userAddress != address(0), "Invalid user address");

        require(relayerAddress != address(0), "Invalid relayer address");

        require(proxyAddress != address(0), "Invalid proxy address");

                require(_registeredWallets[userAddress] == address(0), "User already registered"); // Ensures one-time registration


        // Verify signature

        bytes32 messageHash = keccak256(abi.encodePacked("registerContributorViaRelayer", userAddress, relayerAddress, proxyAddress, salt));

                                                bytes32 ethSignedMessageHash = MessageHashUtils.toEthSignedMessageHash(messageHash);

        address recoveredAddress = ECDSA.recover(ethSignedMessageHash, userSignature);

        require(recoveredAddress == userAddress, "Invalid owner signature");
```

```
        // Use the factory to create a new wallet for the user
                address newWallet = _ykyrFactory.createWalletViaRelayer(userAddress,
relayerAddress, proxyAddress, address(_dataRegistry), address(_teePool));

        // Associate the new wallet with the user
        _registeredWallets[userAddress] = newWallet;

        // Emit event for the newly created wallet
        emit YKYRWalletCreated(newWallet, userAddress);
    }

    function createYKYRWallet() external override {
        // Use the factory to create a new wallet for the user
         address newWallet = _ykyrFactory.createWallet(msg.sender,address(_dataRegistry),
address(_teePool));
        emit YKYRWalletCreated(address(newWallet), msg.sender);
    }
```

As observed, these functions call the `createWalletViaRelayer` and `createWallet` functions in the `YKYRWalletFactory` contract.

```
     function createWallet(address _owner, address _dataRegistry, address _teePool)
external returns (address) {
        require(_dataRegistry != address(0), "Invalid DataRegistry address");
        require(_teePool != address(0), "Invalid TeePool address");
        address clone = Clones.clone(implementation);
        YKYRWalletImp(clone).initialize(_owner, _dataRegistry, _teePool);
        emit WalletCreated(clone);
        return clone;
    }

    // Create wallet for relayer interaction
      function createWalletViaRelayer(address _owner, address _relayerAccount, address
_proxyAccount, address _dataRegistry, address _teePool) external returns (address) {
        require(_owner != address(0), "Invalid owner address");
        require(_relayerAccount != address(0), "Relayer cannot be zero address");
```

```
        require(_proxyAccount != address(0), "Proxy cannot be zero address");
        require(_dataRegistry != address(0), "Invalid DataRegistry address");
        require(_teePool != address(0), "Invalid TeePool address");


        address clone = Clones.clone(implementation); // Create wallet clone
             YKYRWalletImp(clone).initializeWithRelayerAndProxy(_owner, _relayerAccount,
_proxyAccount, _dataRegistry, _teePool); // Initialize with relayer and proxy
        emit WalletCreatedViaRelayer(clone, _owner, _proxyAccount);
        return clone;
    }
```

As observed, these functions lack any access control, allowing anyone to call them. A potential vulnerability arises since these functions emit events for wallet creation. In an example scenario, an attacker can front-run a legitimate `registerContributorViaRelayer` call to call `createWalletViaRelayer` with malicious inputs resulting in this event emission being picked up by the off-chain systems and users expecting a wallet creation, misleading them into thinking this wallet is the wallet they created.

Impact

Users and off-chain systems will be manipulated into accepting the malicious wallet as the wallet that was created for the user.

Recommendation

Implement access control into the `createWalletViaRelayer` and `createWallet` functions in the `YKYRWalletFactory` contract so that only whitelisted addresses such as the `YKYRDLP` contract can use these functions.

Alternatively, in these events, emit the caller so that the off-chain systems can recognize which event is legitimate.

Status

Resolved

Low

[L-01] YKYRDLP, YKYRWalletFactory - Missing checks for `address(0)` in constructor/initializers

Description

It is good practice to implement checks for `address(0)` to prevent accidental inputs that can lead to the need for redeploying contracts. Instances are shown below.

```
implementation = _implementation; //YKYRWalletFactory
_ykyrFactory = YKYRWalletFactory(params.ykyrFactoryAddress); //YKYRDLP
_dataRegistry = IDataRegistry(params.dataRegistryAddress); //YKYRDLP
_teePool = ITeePool(params.teePoolAddress); //YKYRDLP
_transferOwnership(params.ownerAddress); //YKYRDLP
```

Recommendation

It is strongly recommended to implement checks to prevent the zero address from being set during the initialization of contracts. This can be achieved by adding required statements that ensure address parameters are not the zero address. An example is shown below.

```
  constructor(address _implementation) {
       require(_implementation != address(0), "Invalid address");
       implementation = _implementation;
   }
```

Status

Resolved

[L-02] YKYRDLP#updateDataRegistry, updateTeePool, updateYKYRFactory - Missing checks for `address(0)` when updating state variables

Description

It is good practice to implement checks for `address(0)` to prevent accidental inputs that can lead to unexpected functionality.

Recommendation

It is strongly recommended to implement checks to prevent the zero address from being set with these functions. This can be achieved by adding required statements that ensure address parameters are not the zero address. An example is shown below.

```
   function updateDataRegistry(address newDataRegistry) external override onlyOwner {
        if(newDataRegistry == address(0)) {
            revert InvalidDataRegistry();
        }
        _dataRegistry = IDataRegistry(newDataRegistry);
    }
```

Status

Resolved

[L-03] YKYRWalletImp - Use `Ownable2StepUpgradeable` rather than `OwnableUpgradeable`

Description

`Ownable2StepUpgradeable` prevents the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using `Ownable2StepUpgradeable` from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental

transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Resolved

[L-04] YKYRDLP - Upgradeable contract is missing `__gap[..]` storage variable

Description

Storage gaps are needed to not break storage layouts when adding new variables to base contracts. Listed contracts may not be inherited but it is good practice to add them now rather than forgetting later.

Recommendation

When designing upgradeable contracts, it's important to include `__gap[..]` storage variables as storage gaps to prevent issues with storage layout changes when adding new variables to base contracts in the future. While the listed contracts may not be directly inherited from, it is a good practice to add `__gap[..]` storage variables now to ensure a robust and future-proof upgradeable contract design. This proactive approach helps avoid potential complications and storage layout conflicts in the later stages of contract development.

Status

Resolved

[L-05] YKYRDLP, YKYRWalletImp - Implementation contracts lack `_disableInitializers`

Description

The upgradeable contracts do not disable initializers in the constructor, as recommended by the imported Initializable contract. This means that anyone can call the initializer on the implementation contract to set the contract variables and assign

the roles. To reduce the potential attack surface, `_disableInitializers` in the constructor need to be called.

Recommendation

Build a constructor function in the upgradeable contracts that calls the `_disableInitializers` function.

Status

Resolved

[L-06] Contracts - Use of floating pragma

Description

Contracts use a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider known bugs for the compiler version that is chosen.

Status

Acknowledged

[L-07] YKYRDLP#addRewardsForContributors, updateDataRegistry, updateTeePool, updateYKYRFactory - Important functions lack event emissions

Description

Critical functions should emit events to provide transparency and enable off-chain monitoring of important state changes. Without events, it becomes difficult to track

and verify contract operations, complicating both user interfaces and protocol monitoring.

Recommendation

Review your contracts to implement event emissions for critical state variable updates.

Status

Resolved

Gas

[G-01] YKYRDLP, YKYRWalletImp - Prevent setting a state variable with the same value

Description

Not only is wasteful in terms of gas, but this is especially problematic when an event is emitted and the old and new values set are the same, as listeners might not expect this kind of scenario. Instances are shown below.

```
function updateDataRegistry() //YKYRWalletImp
function updateTeePool() //YKYRWalletImp
function updateFileRewardFactor() //YKYRDLP
function updateDataRegistry() //YKYRDLP
function updateTeePool() //YKYRDLP
function updateProofInstruction() //YKYRDLP
function updateMasterKey() //YKYRDLP
function updateYKYRFactory() //YKYRDLP
```

Recommendation

Implement checks in your functions to avoid setting state variables to their existing values. Prior to updating a state variable, compare the new value with the current value and proceed with the assignment only if they differ. An example is shown below.

```
    function updateFileRewardFactor(uint256 newFileRewardFactor) external override
onlyOwner {
        if (_fileRewardFactor == newFileRewardFactor) {
            revert FileRewardFactorNotChanged();
        }
        _fileRewardFactor = newFileRewardFactor;
        emit FileRewardFactorUpdated(newFileRewardFactor);
    }
```

Status

Resolved

[G-02] YKYRDLP#createYKRWallet - Unnecessary casting as a variable is already of the same type

Description

Explicitly casting a variable to a type that it already represents is redundant and can lead to confusion and clutter in the code. This unnecessary casting doesn't typically consume additional gas since Solidity's optimizer often removes such redundant conversions during compilation. However, it does affect code readability and may obscure the actual intent of the code, making it harder for developers to understand and maintain. Ensuring that casting is used only when necessary helps maintain clean, clear, and efficient code.

Recommendation

Update the function to avoid unnecessary casting. An example is shown below.

```
   function createYKYRWallet() external override {
        // Use the factory to create a new wallet for the user
         address newWallet = _ykyrFactory.createWallet(msg.sender,address(_dataRegistry),
address(_teePool));
        emit YKYRWalletCreated(newWallet, msg.sender);
    }
```

Status

Resolved

[G-03] YKYRWalletImp - Revert string sizes are not optimized

Description

Shortening the revert strings to fit within 32 bytes will decrease deployment time and decrease runtime gas when the revert condition is met.

Revert strings that are longer than 32 bytes require at least one additional `mstore`, along with additional overhead to calculate memory offset, etc. Instances of this are shown below.

```
Line 31: require(msg.sender == proxyAccount, "Only proxy can call this function");
//String length is `33`


Line 36: require(msg.sender == relayerAccount, "Only relayer can call this function");
//String length is `35`


Line 171: require(_rewardAddress != address(0), "Reward address cannot be zero address");
//String length is `37`
```

Recommendation

To optimize Gas usage in your contract, it is recommended to keep revert strings as short as possible and to ensure that they fit within 32 bytes. It is possible to use abbreviations or simplified error messages to keep the string length short. Doing so can reduce the amount of Gas used during deployment and runtime when the revert condition is met.

Status

Resolved

[G-04] YKYRDLP#requestReward - Avoid repeating calculations

Description

Repeating the same computations within a contract can lead to unnecessary gas consumption and reduce the contract's efficiency. This is particularly relevant in functions that are called frequently or involve complex calculations. Repeating computations not only wastes computational resources but also increases the cost of executing transactions.

```
    function requestReward(uint256 fileId, uint256 proofIndex) external override
whenNotPaused nonReentrant {
        // rest of the function
        uint256 product = _fileRewardFactor * fileProof.data.score;

        file.rewardAmount = product / 1e18;

        file.timestamp = block.timestamp;
        file.proofIndex = proofIndex;
        file.rewardAmount = (_fileRewardFactor * fileProof.data.score) / 1e18;
        // rest of the function
```

Recommendation

Change the function in order to avoid doing the same calculation. An example is shown below.

```
    function requestReward(uint256 fileId, uint256 proofIndex) external override
whenNotPaused nonReentrant {
        // rest of the function
        uint256 product = _fileRewardFactor * fileProof.data.score;

        file.rewardAmount = product / 1e18;
        file.timestamp = block.timestamp;
        file.proofIndex = proofIndex;
        // rest of the function
```

Status

Resolved

[G-05] YKYRWalletImp, YKYRDLPStorageV1 - Unused state variables

Description

There are state variables that are declared but not used in any function. This might increase the contract's gas usage unnecessarily.

```
uint256 points; //YKYRWalletImp
address internal _payMaster; //YKYRDLPStorageV1
```

Recommendation

To optimize your contract's gas usage, remove any state variables that are declared but not used in any function. Unnecessary state variables can increase gas costs and should be eliminated during code review.

Status

Resolved

[G-06] YKYRWalletFactory#implementation - State variables only set in the constructor should be declared immutable

Description

Declaring state variables that are only set in the constructor as immutable saves gas.

```
address public implementation;
```

Recommendation

Declare the state variable as immutable.

Status

Resolved

QA

[Q-01] YKYRDLP#requestReward - The `nonReentrant` modifier should occur before all other modifiers

Description

In Solidity, the order in which modifiers are applied to a function can significantly impact the function's behavior and security. The `nonReentrant` modifier is crucial for preventing reentrancy attacks, a common vulnerability in smart contracts. This modifier should be the first in the list of applied modifiers to ensure that the reentrancy check is performed before any other logic in the function. Placing `nonReentrant` after other modifiers could potentially lead to security weaknesses, as other modifiers might perform state changes or external calls before the reentrancy protection takes effect. Therefore, correct positioning of the `nonReentrant` modifier is essential for maintaining the integrity and security of the function and, by extension, the entire contract.

Recommendation

To ensure the effectiveness of the `nonReentrant` modifier in protecting against reentrancy, it should be placed as the first modifier in a function's list of modifiers, before all other modifiers. This helps prevent reentrancy attacks from being triggered by other modifiers.

Status

Resolved

[Q-02] YKYRDLP, YKYRWalletImp - Contract uses both `require()/revert()` as well as custom errors

Description

Consider using just one method in a single file to improve code readability and clarity.

Recommendation

Consistently use either `require()/revert()` or custom errors for error handling in your contract to maintain code consistency and readability. Using both methods in the same contract can lead to confusion and make the code harder to understand.

Status

Acknowledged

[Q-03] YKYRDLP, YKYRWalletImp - Constants in comparisons should appear on the left side

Description

Doing so will prevent typo bugs.

Recommendation

To prevent typo bugs and improve code readability, it's advisable to place constants on the left side of comparisons. This coding practice helps catch accidental assignment (=) instead of comparison (==) and enhances code quality.

Status

Resolved

Centralisation

The YKYR project values community-centered and transparency over centralisation.

YKYR emphasizes a community-centric approach while maintaining transparency. Although some critical functions remain centralized for enhanced security and functionality, the project strives to balance decentralization with internal team accountability. This blend ensures operational security while keeping the protocol aligned with decentralized values.

Conclusion

After Hashlock's analysis, the YKYR project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au